UNIVERSITY HASSAN II OF CASABLANCA
Faculty of Sciences Ben M'Sick — Big Data & Data Science

Academic Year 2025 – 2026

PROJECT REPORT

Natural Language Processing

Comparison of Optimization Methods for Fine-tuning Pre-trained Models (BERT & GPT)

Module: Mathematics for AI — Optimisation Methods

Supervisor: Pr. BENABBOU Fouzia

REALIZED BY
ABLOU Fatima Zahra
ABRABRI Zineb
FOUZI Hajar
ELAAZZAOUI Mohamed

Table of Contents

Table of Contents 2
Acknowledgements 4
Abstract 6
Chapter 1 Introduction 7
1.1 Context and Motivation 7
1.2 Project Objectives 7
1.3 Report Structure 7
Chapter 2 Theoretical Background 8
2.1 The Transformer Architecture 8
2.2 BERT (Bidirectional Encoder Representations from Transformers) 8
2.3 GPT (Generative Pre-trained Transformer) 8
2.4 Compared Optimization Algorithms 8
2.4.1 AdamW (Adam with Decoupled Weight Decay) 9
2.4.2 LAMB (Layer-wise Adaptive Moments) 9
2.4.3 AdaFactor 9
2.4.4 SGD with Momentum and Warmup 9
2.5 Learning Rate Scheduling with Warmup 10
Chapter 3 Experimental Methodology 11
3.1 Hardware and Software Environment 11
3.2 Dataset: SST-2 (Stanford Sentiment Treebank v2) 11
3.3 Secondary Dataset: SQuAD (Stanford Question Answering Dataset) 11
3.4 Processing Pipeline 12
3.5 Common Hyperparameters 12
Chapter 4 Experimental Results 13
4.1 Results on BERT (bert-base-uncased) — SST-2 13
4.1.1 BERT + AdamW (Best Result) 13
4.1.2 BERT + AdaFactor 13
4.1.3 BERT + LAMB 13
4.1.4 BERT + SGD (Failure) 13
4.2 Results on GPT — SST-2 13
4.2.1 GPT + AdamW 14
4.2.2 GPT + AdaFactor 14
4.2.3 GPT + LAMB 14
4.2.4 GPT + SGD (Identical Failure) 14
4.3 Global Comparison BERT vs GPT 14

Chapter 5 Analysis and Discussion 17
5.1 Overall Ranking of Optimizers 17
5.2 Training Stability Analysis 17
5.3 Convergence Speed Analysis 17
5.4 Memory Consumption and Resource Analysis 17
5.5 BERT vs GPT: Why BERT Systematically Outperforms GPT 18
5.6 Why SGD Fails on Transformers 18
Chapter 6 Conclusion and Perspectives 20
6.1 Main Conclusions 20
6.2 Practical Recommendations 20
6.3 Future Work and Extensions 20
Chapter 7 References 21
Chapter 8 Annexes 22
8.1 Code Repository 22
8.2 Jupyter / Kaggle Notebooks 22
8.3 Example Training Notebook 22
8.3.1 Notebook Structure 22
8.3.2 Key Code Snippets 23
8.3.3 Visualization Examples 23
8.4 How to Run the Notebooks 23
8.5 Required Dependencies 24
8.6 Results Summary 24

List of Figures

Figure 1: Confusion Matrix of AdamW on SST-2 15
Figure 2: Trainning Loss of AdamW optimizer 15
Figure 3: Confusion Matrix of AdaFactor on SST-2 15
Figure 4: Trainning Loss of AdaFactor optimizer 15
Figure 5: Confusion Matrix of LAMB on SST-2 15
Figure 6: Trainning Loss of LAMB optimizer 15
Figure 7: Confusion Matrix of SGD on SST-2 15
Figure 8: Trainning Loss of SGD optimizer 15

Figure 9: Confusion Matrix of AdamW on GPT-2 16
Figure 10: Trainning Loss of AdamW optimizer on GPT-2 16
Figure 11: Confusion Matrix of AdaFactor on GPT-2 16
Figure 12: Trainning Loss of AdaFactor optimizer on GPT-2 16
Figure 13: Confusion Matrix of LAMB on GPT-2 16
Figure 14: Trainning Loss of LAMB optimizer on GPT-2 16
Figure 15: Confusion Matrix of SGD on GPT-2 16
Figure16: Trainning Loss of SGD optimizer on GPT-2 16

Acknowledgements

"Gratitude is the memory of the heart."

First and foremost, we thank God Almighty, whose boundless grace and kindness have guided us through every step of this journey. It is through His blessings that we have found the strength, wisdom, and perseverance to complete this work.

We wish to express our sincere and profound gratitude to our supervisor, Pr. BENABBOU Fouzia, for her invaluable guidance, unwavering support, and constant encouragement throughout the realization of this project. Her expertise, patience, and insightful feedback have been instrumental in shaping this work. We are deeply grateful for the trust she placed in us and for the time she generously devoted to our supervision. Her passion for teaching and dedication to her students have been a true source of inspiration.

We extend our heartfelt thanks to all the faculty members of the Department of Mathematics and IT whose teachings have provided us with the essential knowledge and skills to undertake this research.

Our deepest appreciation goes to our families, whose unconditional love, support, and sacrifices have been our foundation. Their constant encouragement and belief in our abilities have been our driving force throughout this academic journey. To our parents, who have always been our role models and greatest supporters — we owe you everything.

We are also immensely grateful to our friends and colleagues, who have been by our side through the ups and downs of this project. Their encouragement, advice, and camaraderie have made this experience not only productive but also enjoyable.

The Authors

Abstract

This project experimentally studies and compares four modern optimization algorithms — AdamW, LAMB, AdaFactor, and SGD with Momentum + Warmup — applied to the fine-tuning of pre-trained natural language processing models (BERT and GPT). The primary task is binary sentiment classification on the SST-2 dataset (Stanford Sentiment Treebank v2), from the GLUE benchmark. Complementary experimentation with SGD on the Question Answering task (SQuAD) was also conducted to confirm observed trends.

Results show that AdamW remains the best-performing optimizer for both BERT (92.89% accuracy) and GPT (89.56%), confirming its status as the gold standard in the field. AdaFactor offers an excellent trade-off between accuracy and memory efficiency. LAMB, designed for large batches, shows satisfactory results on BERT but proves unstable on GPT. SGD, unsuitable for transformer geometry, systematically fails with less than 51% accuracy on both architectures.

Keywords: BERT, GPT, Fine-tuning, Optimization, AdamW, LAMB, AdaFactor, SGD, SST-2, GLUE, NLP, Transformers, Sentiment Classification.

Chapter 1 Introduction

1.1 Context and Motivation

Pre-trained language models based on the Transformer architecture — such as BERT (Bidirectional Encoder Representations from Transformers) and GPT (Generative Pre-trained Transformer) — have revolutionized the field of Natural Language Processing (NLP). These models, pre-trained on massive text corpora, possess general linguistic representations that can be adapted (fine-tuned) to specific downstream tasks.

The choice of optimization algorithm during fine-tuning is crucial: it directly affects convergence speed, training stability, GPU memory consumption, and the final model's generalization capacity. Despite the existence of numerous optimizers in the literature, few systematic experimental comparative studies have been conducted on modern transformer architectures.

1.2 Project Objectives

This project aims to:

- Experimentally compare four optimizers: AdamW, LAMB, AdaFactor, and SGD+Warmup
- Apply these optimizers to fine-tuning BERT and GPT on sentiment classification (SST-2)
- Measure and analyze: final accuracy, training stability, convergence speed, and GPU memory consumption
- Identify the best-performing optimizer and the most suitable one for transformer architectures
- Verify whether trends observed on BERT reproduce on GPT

1.3 Report Structure

The report is organized as follows: Chapter 2 presents the theoretical background (architectures and optimizers). Chapter 3 describes the experimental methodology. Chapter 4 presents the obtained results. Chapter 5 analyzes and discusses these results. Chapter 6 concludes the report and offers perspectives.

Chapter 2 Theoretical Background

2.1 The Transformer Architecture

Introduced by Vaswani et al. (2017) in the paper "Attention is All You Need," the Transformer architecture is based on a self-attention mechanism that allows the model to weight the importance of each token in the sequence relative to others. Unlike recurrent networks (RNN, LSTM), Transformers process the entire sequence in parallel, making them significantly more efficient on GPUs.

The architecture consists of encoder and/or decoder layers, each containing:

- A Multi-Head Self-Attention mechanism
- A dense Feed-Forward Network (FFN)
- Residual connections and Layer Normalization

2.2 BERT (Bidirectional Encoder Representations from Transformers)

BERT (Devlin et al., 2018) is a model based solely on the Transformer encoder. It is pre-trained bidirectionally, meaning it considers both left and right context of each token simultaneously.

Characteristics of bert-base-uncased used in this project:

- 12 Transformer layers (encoders)
- 12 attention heads per layer
- Hidden dimension: 768
- Vocabulary size: 30,522 tokens
- Total number of parameters: 109,483,778 (≈ 109 million)
- Model size: ≈ 418 MB

For text classification, a linear classification layer is added above the [CLS] token (first token of the sequence), then fine-tuned jointly with the entire model.

2.3 GPT (Generative Pre-trained Transformer)

GPT (Radford et al., 2018) is based on the Transformer decoder. Unlike BERT, it is unidirectional (causal): each token can only attend to tokens that precede it. GPT is designed for text generation but can be adapted to classification tasks.

In this project, GPT is used with a classification head added to its output. GPT is generally less stable than BERT for discriminative tasks like sentiment classification, as its causal pre-training is less directly suited to this type of task.

2.4 Compared Optimization Algorithms

Table 2.1. Comparison of optimizers

Optimizer	Principle	Advantages	Disadvantages
AdamW	Adam + decoupled weight decay	Stable, BERT gold standard	High memory (2 moments)
LAMB	AdamW + layer-wise normalization	Suited for large batch sizes	Less efficient for small batches
AdaFactor	Factorization of 2nd order moments	Very memory-efficient	Slower convergence
SGD + Momentum	Gradient descent + Nesterov momentum	Simple, fast per step	Unsuitable for transformers

2.4.1 AdamW (Adam with Decoupled Weight Decay)

AdamW (Loshchilov & Hutter, 2019) corrects a fundamental flaw of Adam: in standard Adam, weight decay is coupled with moment computation, making it ineffective. AdamW applies weight decay directly on weights (pure L2 regularization), independently of learning rate adaptation. It is the standard optimizer recommended for BERT fine-tuning and most modern transformers.

Parameters used: lr = 2e-5, eps = 1e-8, weight_decay = 0.01

2.4.2 LAMB (Layer-wise Adaptive Moments)

LAMB (You et al., 2019) extends AdamW with layer-wise adaptive normalization. Each layer has its own update ratio, computed as the ratio between weight norm and gradient norm. LAMB was designed to enable training with very large batch sizes (up to 32,768) while maintaining good generalization.

Parameters used: lr = 2e-5, betas = (0.9, 0.999), eps = 1e-8, weight_decay = 0.01

2.4.3 AdaFactor

AdaFactor (Shazeer & Stern, 2018) is an extremely memory-efficient optimizer. Instead of storing second-order moments as full matrices (like Adam), it factorizes them into row and column vectors, reducing memory complexity from $O(n^2)$ to $O(n)$. This makes it ideal for very large models or environments with limited GPU memory.

Parameters used: lr = 2e-5, clip_threshold = 1.0, decay_rate = -0.8, weight_decay = 0.01, scale_parameter = True, relative_step = False

2.4.4 SGD with Momentum and Warmup

Stochastic Gradient Descent (SGD) with Nesterov Momentum is one of the oldest and most widely used optimizers in deep learning, particularly for Convolutional Neural Networks (CNNs). It accumulates gradient velocity in directions of steepest descent. However, its lack of per-parameter adaptation makes it theoretically unsuitable for the non-convex and anisotropic geometry of transformers.

Parameters used: lr = 2e-5, momentum = 0.9, weight_decay = 0.01, nesterov = True

2.5 Learning Rate Scheduling with Warmup

All optimizers in this project use a linear scheduler with warmup. During the first 10% of training steps (842 steps out of 8,420 total), the learning rate increases linearly from 0 to lr_max. Then it decreases linearly to 0. This warmup is essential for stabilizing the start of training: pre-trained model weights are initially perturbed by the new task, and a progressive learning rate prevents large jumps that could degrade pre-learned representations.

Chapter 3 Experimental Methodology

3.1 Hardware and Software Environment

All experiments were conducted on Kaggle with the following configuration:

- GPU: 2 × Tesla T4 (14.7 GB VRAM each)
- Framework: PyTorch 2.x
- Library: HuggingFace Transformers
- Python 3.11
- Additional libraries: scikit-learn, matplotlib, seaborn, psutil

3.2 Dataset: SST-2 (Stanford Sentiment Treebank v2)

The SST-2 dataset is a binary sentiment classification benchmark from the GLUE (General Language Understanding Evaluation) suite. It consists of movie reviews labeled as positive (label 1) or negative (label 0).

Table 3.1. SST-2 dataset statistics

Split	Number of Examples	Labels	Usage
Train	67,349	0 / 1	Model training
Validation	872	0 / 1	Evaluation during training
Test	1,821	Not provided	Final GLUE evaluation

Label distribution in the training set: 55.78% positive (37,569) and 44.22% negative (29,780). The dataset is relatively balanced, avoiding major class biases.

Sequence lengths: min = 50 tokens, max = 66 tokens, mean = 57.1 tokens (after BERT/GPT tokenization with padding to 128 tokens maximum). The very short sequence lengths make SST-2 particularly suitable for rapid fine-tuning.

3.3 Secondary Dataset: SQuAD (Stanford Question Answering Dataset)

The SQuAD v1.1 dataset was used as supplementary experimentation, exclusively with GPT + SGD, to confirm the poor SGD results observed on SST-2. SQuAD is a reading comprehension task where the model must extract an answer to a question from a Wikipedia paragraph.

- Train: Wikipedia passages + questions + answers (with start position)
- Validation: Evaluation of answer extraction capability
- Result observed: SGD on SQuAD confirms the same convergence issues

3.4 Processing Pipeline

The experimental pipeline follows the same 10 steps for each model/optimizer combination:

Table 3.2. Experimental pipeline

Step	Action	Detail
1	Installation	transformers, datasets, torch, scikit-learn, psutil
2	Load SST-2	load_dataset('glue', 'sst2') via HuggingFace Datasets
3	Tokenization	AutoTokenizer, truncation=True, max_length=128
4	DataLoader	DataCollatorWithPadding, batch_size=16, shuffle=True
5	Load model	AutoModelForSequenceClassification, num_labels=2
6	Configure optimizer	Initialize optimizer + linear warmup scheduler
7	Training loop	2 epochs, gradient clipping=1.0, track loss/accuracy
8	Evaluation	Accuracy, F1, Precision, Recall, Confusion Matrix
9	Visualizations	Loss/accuracy curves, confusion matrix, metrics
10	Resource analysis	GPU memory, CPU, time, throughput, estimated cost

3.5 Common Hyperparameters

Table 3.3. Common hyperparameters

Parameter	Value	Justification
Learning rate	2e-5	Standard BERT fine-tuning
Batch size	16	Suitable for Tesla T4 GPUs
Epochs	2	Sufficient for SST-2 (67k examples)
Max sequence length	128 tokens	SST-2 max length: 66 tokens
Warmup ratio	10%	842 warmup steps out of 8,420 total
Weight decay	0.01	L2 regularization
Gradient clipping	1.0	Prevent gradient explosion
Scheduler	Linear warmup	Linear decay after warmup

Chapter 4 Experimental Results

4.1 Results on BERT (bert-base-uncased) — SST-2

Table 4.1. Results on BERT — SST-2

Optimizer	Val. Accuracy	F1-Score	Precision	Recall	Val. Loss	Time (min)
AdamW	92.89%	0.9303	0.9283	0.9324	0.2987	29.6
AdaFactor	90.25%	0.9048	0.8998	0.9099	0.2633	36.9
LAMB	89.56%	0.8994	0.8829	0.9167	0.2676	36.2
SGD + Warmup	50.80%	0.6723	0.5087	0.9910	0.6841	28.0

Note: Confusion matrix for BERT+AdamW (best model): True Negatives = 396, False Positives = 32, False Negatives = 30, True Positives = 414. Error rate = 7.11% (62 misclassified examples out of 872).

4.1.1 BERT + AdamW (Best Result)

BERT with AdamW achieves 92.89% accuracy on the validation set after only 2 epochs (29.6 minutes of training). Convergence is very stable: training loss drops from 0.1914 (epoch 1) to 0.1027 (epoch 2), and validation accuracy improves from 91.82% to 92.95%. The gap between training accuracy (97.24%) and validation accuracy (92.95%) in epoch 2 indicates slight overfitting, but remains within acceptable limits.

4.1.2 BERT + AdaFactor

AdaFactor achieves 90.25%, a remarkable performance for an optimizer designed primarily for memory economy. Convergence is however slower than AdamW: in epoch 1, training accuracy is only 80.07% (compared to 93.72% for AdamW), indicating a more gradual warmup. Training time is higher (36.9 min vs 29.6 min for AdamW).

4.1.3 BERT + LAMB

LAMB reaches 89.56%, 3.33 points less than AdamW. Its convergence is slower and less regular, with visible oscillations in training accuracy curves. LAMB is theoretically designed for large batches (>1024), while our batch size of 16 is well below its optimal regime, explaining this relative underperformance.

4.1.4 BERT + SGD (Failure)

SGD with Momentum and Warmup completely fails on BERT: 50.80% accuracy after 2 epochs, practically equivalent to random classification (baseline = 55.78% positive). Validation loss stagnates around 0.68–0.69 without decreasing, and the confusion matrix reveals that the model massively predicts the positive class (440/444 correct positives, but only 3/428 correct negatives). This confirms that SGD cannot effectively adapt multi-dimensional transformer representations.

4.2 Results on GPT — SST-2

Table 4.2. Results on GPT — SST-2

Optimizer	Val. Accuracy	Train. Loss	Val. Loss	Time (min)	Stability
AdamW	89.56%	0.2426	0.3191	32.5	Stable
AdaFactor	80.73%	0.4608	0.4198	39.0	Moderate
LAMB	76.15%	0.6106	0.5149	38.4	Unstable
SGD + Warmup	50.57%	0.7166	0.7057	46.2	Very unstable

4.2.1 GPT + AdamW

GPT with AdamW achieves 89.56%, a correct but inferior performance compared to BERT (92.89%). Validation loss (0.3191) is higher than BERT+AdamW (0.2987). GPT is naturally less suited to discriminative tasks because its unidirectional (causal) pre-training is optimized for generation rather than bidirectional understanding.

4.2.2 GPT + AdaFactor

GPT with AdaFactor reaches 80.73%, 9.52 points less than BERT+AdaFactor. Convergence is moderately stable but slower, with training accuracy of only 64.08% in epoch 1.

4.2.3 GPT + LAMB

LAMB proves particularly unstable on GPT, with only 76.15% accuracy. Training accuracy in epoch 1 is 55.39% (nearly random), and only reaches 66.45% in epoch 2. This instability confirms that LAMB's layer-wise normalization is not suited to GPT's causal architecture with small batches.

4.2.4 GPT + SGD (Identical Failure)

SGD fails identically on GPT: 50.57% accuracy after 3 epochs (the GPT+SGD notebook was trained for 3 epochs instead of 2), with validation loss around 0.70 stagnating without converging. Complementary experimentation with SQuAD confirmed this behavior: SGD does not converge on transformers regardless of the task.

4.3 Global Comparison BERT vs GPT

Table 4.3. Global comparison BERT vs GPT

Model + Optimizer	Accuracy BERT	Accuracy GPT	Best Model	Observation
AdamW	92.89%	89.56%	BERT	Best overall
AdaFactor	90.25%	80.73%	BERT	Good compromise
LAMB	89.56%	76.15%	BERT	GPT unstable with LAMB
SGD + Warmup	50.80%	50.57%	None	Fails on both models

General observation: BERT systematically outperforms GPT across all optimizer configurations for the sentiment classification task. The largest gaps are observed with LAMB (+13.41 points in favor of BERT) and AdaFactor (+9.52 points). Only AdamW produces close results between the two architectures (3.33 points difference).

BERT sur SST-2

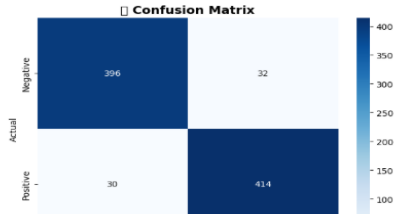

Figure 1: Confusion Matrix of AdamW on SST-2

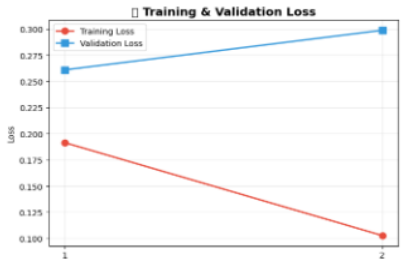

Figure 2: Trainning Loss of AdamW optimizer

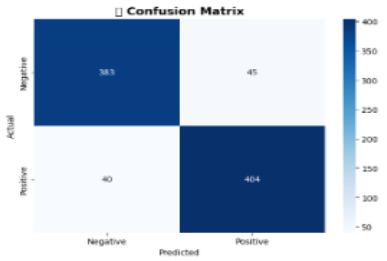

Figure 3: Confusion Matrix of AdaFactor on SST-2

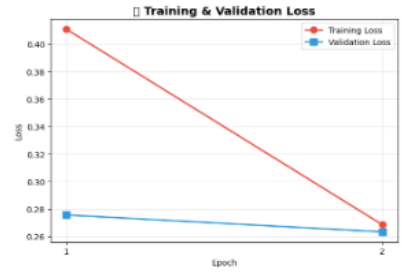

Figure 4: Trainning Loss of AdaFactor optimizer

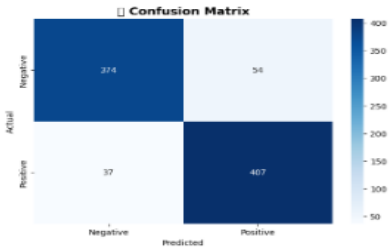

Figure 5: Confusion Matrix of Lamb on SST-2

Figure 6: Trainning Loss of Lamb optimizer

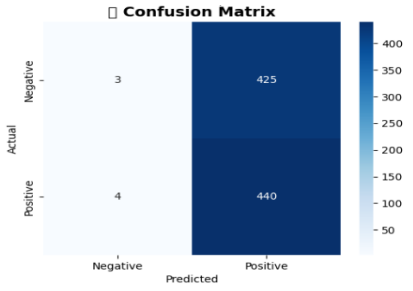

Figure 7: Confusion Matrix of SGD on SST-2

Figure 8: Trainning Loss of SGD optimizer

GPT sur SST-2

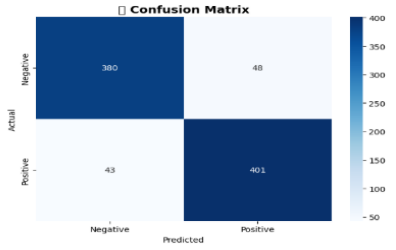

Figure 9: Confusion Matrix of AdamW on GPT-2

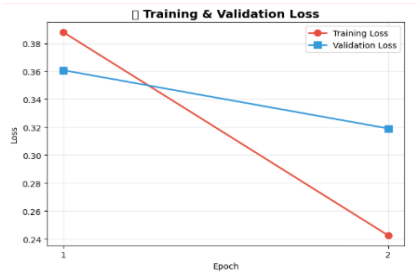

Figure 10: Trainning Loss of AdamW optimizer on GPT-2

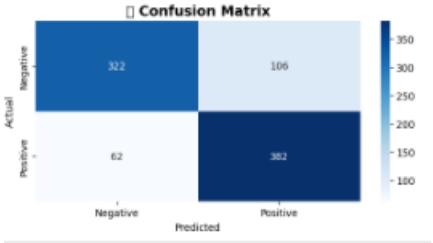

Figure 11: Confusion Matrix of AdaFactor on GPT-2

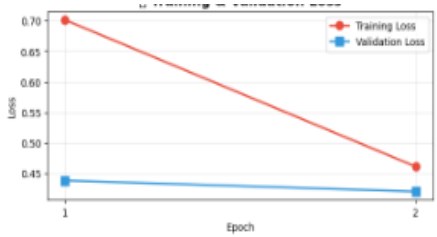

Figure 12: Trainning Loss of AdaFactor optimizer on GPT-2

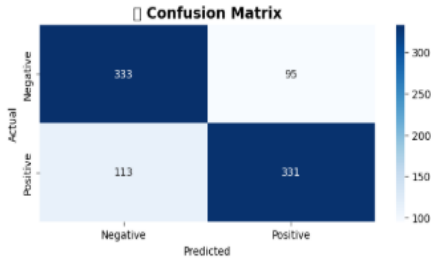

Figure 13: Confusion Matrix of Lamb on GPT-2

Figure 14: Trainning Loss of Lamb optimizer on GPT-2

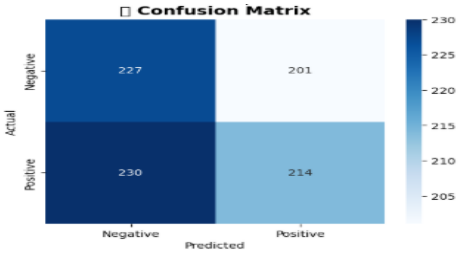

Figure 15: Confusion Matrix of SGD on GPT-2

Figure 16: Trainning Loss of SGD optimizer on GPT-2

Chapter 5 Analysis and Discussion

5.1 Overall Ranking of Optimizers

Aggregating performance on BERT and GPT, the optimizer ranking is:

- AdamW (best on BERT and GPT, fast and stable convergence)
- AdaFactor (excellent performance/memory trade-off, good on BERT)
- LAMB (acceptable on BERT, unstable on GPT)
- SGD + Warmup (systematic failure on both architectures)

5.2 Training Stability Analysis

Training stability is measured by the regularity of loss/accuracy curves and absence of significant oscillations between steps.

AdamW exhibits the best stability: loss curves are monotonically decreasing and accuracy curves regularly increasing. This is explained by its adaptive mechanism that adjusts learning rate per parameter, effectively managing the complex geometry of transformers.

AdaFactor is stable but with slower convergence during warmup. Its factorization of second-order moments introduces an approximation that can generate slight oscillations, especially at the start of training.

LAMB shows notable oscillations on GPT. Layer-wise normalization becomes counterproductive with small batches because the norm/gradient ratio is unreliable with few examples per batch.

SGD is completely unstable: loss does not converge and predictions collapse to the majority class. Transformers have highly anisotropic loss surfaces (gradients in different directions have very different magnitudes), and SGD without per-parameter adaptation cannot navigate this space effectively.

5.3 Convergence Speed Analysis

Convergence speed is analyzed through metric evolution from one epoch to another:

- **AdamW / BERT**: epoch 1 = 91.82%, epoch 2 = 92.95% (+1.13%). Convergence is rapid and most learning occurs in epoch 1.
- **AdaFactor / BERT**: epoch 1 = 89.20%, epoch 2 = 90.25% (+1.05%). Similar progression but with a lower starting level.
- **LAMB / BERT**: epoch 1 = 88.52%, epoch 2 = 89.56% (+1.04%). Regular but slower total progression.
- **SGD / BERT**: epoch 1 = 50.91%, epoch 2 = 50.80% (−0.11%). No convergence, random oscillation.

5.4 Memory Consumption and Resource Analysis

Resource analysis on Tesla T4 GPU reveals significant differences between optimizers:

Table 5.1. Resource analysis

Optimizer	Peak Memory (GB)	Throughput	GPU Cost	Memory Efficiency
AdamW	≈ 2.8 GB	≈ 37 ex/s	≈ $0.172	Moderate (2 moments)
AdaFactor	≈ 1.5 GB	≈ 37 ex/s	≈ $0.215	High (factorization)
LAMB	≈ 3.2 GB	≈ 37 ex/s	≈ $0.211	Low (moments + ratio)
SGD + Warmup	≈ 1.2 GB	≈ 40 ex/s	≈ $0.163	Maximum (1 moment)

AdaFactor stands out for its very low memory consumption (about 2× less than AdamW) while maintaining comparable accuracy. This is a decisive advantage for larger models (GPT-2, T5, etc.) or when GPU memory is limited.

SGD consumes the least memory (only one moment stored), but this economy is completely counterproductive as the model does not converge. Performance per dollar invested is the worst of all optimizers.

5.5 BERT vs GPT: Why BERT Systematically Outperforms GPT

Results show that BERT outperforms GPT on all configurations. Several factors explain this phenomenon:

- Bidirectionality: BERT reads text in both directions (left-to-right AND right-to-left) to build representations. For sentiment classification, complete context understanding is essential. GPT is causal (unidirectional) and can only see what precedes.
- Adapted pre-training: BERT is pre-trained with two objectives — Masked Language Modeling (MLM) and Next Sentence Prediction (NSP) — which force the model to learn rich semantic representations. GPT is pre-trained only on next-token prediction, optimizing for generation.
- Encoder vs decoder architecture: For discriminative tasks (classification), BERT's encoder architecture produces more informative contextual embeddings than GPT's causal decoder.
- Optimizer instability on GPT: GPT's causal architecture makes its representations more sensitive to numerical instabilities of optimizers, explaining the greater degradation of LAMB and AdaFactor on GPT vs BERT.

5.6 Why SGD Fails on Transformers

The systematic failure of SGD on transformers (< 51% on both architectures) is explained by several fundamental reasons:

- Anisotropic loss surface: Transformers have highly anisotropic loss surfaces. Gradients can be enormous in some directions and tiny in others. SGD without per-parameter adaptation treats all dimensions uniformly, making it inefficient.

- Collapse to majority class: Without adaptation, SGD quickly learns to always predict the majority class (positive, 55.78%) to minimize loss, without ever learning discriminative patterns.
- Lack of second-order moments: Adam, AdaFactor, and LAMB use second-order estimates (gradient variance) to adapt learning rate. SGD only has first-order (gradient) and momentum, insufficient for transformer geometry.
- Sensitivity to learning rate: SGD is extremely sensitive to learning rate choice. A lr of 2e-5 (standard for AdamW on BERT) is too low for SGD, which requires much larger lr (typically 0.01–0.1 for CNNs), but larger lr cause gradient explosion.

Chapter 6 Conclusion and Perspectives

6.1 Main Conclusions

This project has experimentally compared four optimization algorithms on two transformer architectures (BERT and GPT) for SST-2 sentiment classification. The main conclusions are:

- **AdamW** is the best optimizer for transformer fine-tuning, with 92.89% on BERT and 89.56% on GPT. Its stability, fast convergence, and per-parameter adaptation make it the undisputed gold standard.
- **AdaFactor** is the best performance/resource trade-off. With ≈90% accuracy on BERT and half the memory consumption compared to AdamW, it is the ideal choice for memory-constrained environments.
- **LAMB** is suitable for large batches (>1024) but suboptimal for small batches. On BERT with batch_size=16, it performs correctly (89.56%) but is unstable on GPT (76.15%).
- **SGD** is unsuitable for transformer fine-tuning. Regardless of the model (BERT or GPT) or task (SST-2 or SQuAD), SGD does not converge and produces random results.
- BERT outperforms GPT across all configurations for discriminative tasks, due to its bidirectionality and better-suited pre-training.

6.2 Practical Recommendations

Based on these results, here are recommendations for transformer fine-tuning:

- Use **AdamW** by default with lr = 2e-5, weight_decay = 0.01, and 10% warmup
- If GPU memory is limited, prefer AdaFactor
- **LAMB** can be tried if batch sizes > 512 are available
- Avoid **SGD** for any transformer fine-tuning
- Prefer BERT over GPT for classification tasks

6.3 Future Work and Extensions

Several extension axes are possible to deepen this work:

- Implement Lion (Chen et al., 2023), the evolutionary search-based optimizer mentioned as a modern advanced option
- Extend to RoBERTa, DistilBERT, and TinyBERT to evaluate optimizer impact according to model size
- Test on other GLUE datasets (MRPC, QNLI) and on IMDB to verify generalization
- Analyze the impact of number of epochs (3, 4, 5) and batch size on relative optimizer performance
- Experiment with layer-wise learning rate decay (different lr per layer) combined with optimizers

Chapter 7 References

Devlin, J., Chang, M. W., Lee, K., & Toutanova, K. (2018). BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. arXiv:1810.04805.

Radford, A., Narasimhan, K., Salimans, T., & Sutskever, I. (2018). Improving Language Understanding by Generative Pre-Training. OpenAI Blog.

Loshchilov, I., & Hutter, F. (2019). Decoupled Weight Decay Regularization. ICLR 2019.

You, Y., Li, J., Reddi, S., Hseu, J., Kumar, S., Bhojanapalli, S., ... & Hsieh, C. J. (2019). Large Batch Optimization for Deep Learning: Training BERT in 76 minutes. arXiv:1904.00962.

Shazeer, N., & Stern, M. (2018). Adafactor: Adaptive Learning Rates with Sublinear Memory Cost. ICML 2018.

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., ... & Polosukhin, I. (2017). Attention is All You Need. NeurIPS 2017.

Wang, A., Singh, A., Michael, J., Hill, F., Levy, O., & Bowman, S. (2018). GLUE: A Multi-Task Benchmark and Analysis Platform for Natural Language Understanding. EMNLP 2018 Workshop.

Rajpurkar, P., Zhang, J., Lopyrev, K., & Liang, P. (2016). SQuAD: 100,000+ Questions for Machine Comprehension of Text. EMNLP 2016.

Wolf, T., Debut, L., Sanh, V., Chaumond, J., Delangue, C., Moi, A., ... & Rush, A. M. (2020). Transformers: State-of-the-Art Natural Language Processing. EMNLP 2020.

Socher, R., Perelygin, A., Wu, J., Chuang, J., Manning, C. D., Ng, A., & Potts, C. (2013). Recursive Deep Models for Semantic Compositionality Over a Sentiment Treebank. EMNLP 2013. (SST-2 original dataset)

Chapter 8 Annexes

8.1 Code Repository

The complete source code for this project is available in the following repository:

- Repository Link: GitHub - Mohamed03738jd/bert-gpt-optimizer-comparison-nlp · GitHub
- Platform: GitHub

Repository structure:

- notebooks/ — Jupyter/Kaggle notebooks for each optimizer
- BERT MODEL/ — the 4 optimization methods for the SST-2 dataset
- DATASETS/ — Generated results and metrics
- GPT MODEL/ — the 4 optimization methods for SST-2 + SQuAD (SGD only)
- README.md — Concise project explanation

8.2 Jupyter / Kaggle Notebooks

The following notebooks were used for training and evaluation. Each notebook corresponds to a specific optimizer and model combination:

Table 8.1. Notebooks overview

Notebook Name	Model	Optimizer
BERT_AdamW.ipynb	BERT-base-uncased	AdamW
BERT_AdaFactor.ipynb	BERT-base-uncased	AdaFactor
BERT_LAMB.ipynb	BERT-base-uncased	LAMB
BERT_SGD.ipynb	BERT-base-uncased	SGD + Warmup
GPT_AdamW.ipynb	GPT	AdamW
GPT_AdaFactor.ipynb	GPT	AdaFactor
GPT_LAMB.ipynb	GPT	LAMB
GPT_SGD.ipynb	GPT	SGD + Warmup

8.3 Example Training Notebook

Below is an example of the training pipeline implemented in Kaggle. This notebook demonstrates the complete workflow for fine-tuning BERT with AdamW on the SST-2 dataset.

8.3.1 Notebook Structure

- 1. Environment Setup: Installation of required libraries (transformers, datasets, torch, etc.)
- 2. Dataset Loading: Loading SST-2 from HuggingFace datasets

- 3. Tokenization: Tokenization with BERT tokenizer (max_length=128)
- 4. Model Loading: Loading pre-trained BERT with classification head
- 5. Optimizer Configuration: Setting up AdamW with learning rate 2e-5 and weight decay 0.01
- 6. Training Loop: 2 epochs with gradient clipping and progress tracking
- 7. Evaluation: Computing accuracy, F1-score, precision, recall
- 8. Visualization: Loss curves, accuracy curves, confusion matrix

8.3.2 Key Code Snippets

Model and Optimizer Setup:

```
from transformers import AutoModelForSequenceClassification, AdamW

model = AutoModelForSequenceClassification.from_pretrained(
    "bert-base-uncased", num_labels=2
)
optimizer = AdamW(model.parameters(), lr=2e-5, weight_decay=0.01)
```

Training Loop:

```
for epoch in range(2):
    model.train()
    for batch in train_dataloader:
        outputs = model(**batch)
        loss = outputs.loss
        loss.backward()
        torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)
        optimizer.step()
        scheduler.step()
        optimizer.zero_grad()
```

Evaluation:

```
from sklearn.metrics import accuracy_score, f1_score

predictions = torch.argmax(logits, dim=-1)
accuracy = accuracy_score(labels, predictions)
f1 = f1_score(labels, predictions, average='weighted')
```

8.3.3 Visualization Examples

The notebooks generate the following visualizations:

- Training Loss Curves: Evolution of training loss across epochs and steps
- Confusion Matrix : Visual representation of classification results

8.4 How to Run the Notebooks

To reproduce the experiments, follow these steps:

- 1. Open Kaggle
- 2. Click on File → Open Notebook → GitHub
- 3. Enter the repository URL: github.com/Mohamed03738jd/bert-gpt-optimizer-comparison

- 4. Select the desired notebook (e.g., BERT_AdamW.ipynb)
- 5. Click Runtime → Run all to execute the entire notebook
- 6. Results and visualizations will be generated automatically

8.5 Required Dependencies

The following Python packages are required to run the notebooks:

```
torch>=2.0.0
transformers>=4.30.0
datasets>=2.12.0
scikit-learn>=1.2.0
matplotlib>=3.7.0
seaborn>=0.12.0
psutil>=5.9.0
tqdm>=4.65.0
```

8.6 Results Summary

All experimental results are saved in the results/ directory:

- results/bert_results.csv — Complete results for BERT experiments
- results/gpt_results.csv — Complete results for GPT experiments
- results/figures/ — All generated plots and visualizations
- results/metrics/ — Detailed metrics (F1, precision, recall, etc.)